

ENTREGA COMPORTAMIENTO DE PERSONAJES

ENTREGA FINAL

COYOTE REQUIEM



Eduardo Almarza Blasco - Antonio Bernal de Celis - Diego Fernández Manso -
Candela Jiménez Gonzáles - Andrea Luengo Zazo – David del Castillo Enríquez

ÍNDICE

DESCRIPCIÓN GENERAL	2
ESPECIFICACIONES DE PERSONAJE	4
Enemigo 1: Básico (Melee)	5
Enemigo 2: Dummy (Tutorial)	9
Enemigo 3: Bomba (Melee)	14
Enemigo 4: Heavy (Distancia / Melee)	19
Enemigo 5: Shooter (Distancia)	24
Enemigo 6: Boss	30
INTERACCIONES PERSONAJES – ESCENARIO	37
DESCRIPCIÓN DE ALGORITMOS	41
MUTEX DE ATAQUE	42
FUNCIONAMIENTO DE COBERTURAS.	42
CÍRCULO DE KUNG-FU	43
ACTUALIZACIÓN DE NAVMESH:.....	44
CAMBIOS A LA API.....	45
RESULTADOS.....	49
REPARTO DE TAREAS	54

DESCRIPCIÓN GENERAL

“Coyote Requiem” es un juego de acción en tercera persona que combina el combate frenético y gestión de enemigos de un Hack & Slash con el estilo del boxeo de Punch Out. Este juego es compartido con las demás asignaturas.

El jugador tendrá que atravesar varios niveles repletos de distintos tipos de enemigos, cada uno con su respectivo comportamiento y mecánicas únicas, que se coordinarán para combatir contra el jugador. El juego está principalmente centrado en combates uno contra uno similares a peleas de boxeo, pero también existen enemigos que te atacan desde largas distancias o situaciones donde el jugador tenga que enfrentar varios enemigos al mismo tiempo.

Cada enemigo tiene por tanto varias acciones que puede realizar, simulando diferentes patrones de ataque que combinan movimientos, esquives, golpes, etc. La inteligencia artificial de los enemigos se encargará de decidir los comportamientos de los estos fuera y dentro de los combates: qué acciones tomar, cómo moverse por el mundo, qué secuencias de ataques realizar en función del input del jugador...

Los combates funcionan por **dirección**:

- Un ataque por la derecha solo puede esquivarse o bloquearse por la derecha, y viceversa
- Existen 2 direcciones: izquierda y derecha
- Los enemigos realizan ataques en distintas direcciones, y también se defienden en determinadas direcciones concretas.
- Los enemigos pueden defender por una dirección (dejando la otra desprotegida) pero algunos ataques defenderán ambas direcciones durante un periodo de tiempo, este término será llamado superarmour en el resto de la memoria.
- Si el jugador realiza un esquite en la dirección correcta y en el momento exacto, puede realizar un **esquite perfecto** y contraatacar con un **parry**, que hace daño adicional.

Antes de pelear con un enemigo, tienes que **fijarlo**. Una vez fijado, solo puedes golpear a ese enemigo en particular. Mientras tienes un enemigo fijado, el resto te rodean y te van golpeando por la espalda tomando turnos, es decir, nunca te atacan dos enemigos a la vez. Puedes esquivar **ataques por la espalda**, pero no contraatacarles, salvo que desfijes el objetivo actual y fijas al objetivo de tu espalda. Los ataques a distancia funcionan de la misma manera, pudiendo además ser esquivados, aunque no estés fijando a ningún enemigo.

El jugador tiene también un gancho que le permite:

- Engancharse a cornisas para subir a sitios altos y moverse por el mapa

- Enganchar enemigos, tras lo cual el jugador puede elegir atraer al enemigo o acercarse volando hacia él, siendo ambas útiles para entrar en un rango cuerpo a cuerpo con los enemigos.

Estos serían los controles asociados a las distintas acciones que puede realizar el jugador:

MECÁNICA	Teclado	Dispositivos táctiles	Mando
MOVIMIENTO DE CÁMARA	RATÓN	Joystick táctil der.	Joystick der.
MOVIMIENTO	W,A,S,D	Joystick táctil izq.	Joystick izq.
ENFOCAR	Q	Botón Lockear	RB/R1/Presionar joystick der.
CORRER	SHIFT	Botón correr	RT/R2
ACTIVAR GANCHO	E	Joystick táctil izq.	B/Círculo/Presionar joystick izq.
CAMBIAR OBJETIVO GAN.	A,D	Botón activar gancho	Joystick izq. mover izq. y der.
IR GANCHO	W	Joystick táctil izq.	Joystick izq. arriba
ATRAER GANCHO	S	Joystick táctil izq.	Joystick izq. abajo
LANZAR GANCHO	CLICK IZQ.	Botón lanzar gancho	RT/R2
ATAQUE IZQUIERDO	CLICK IZQ.	Botón ataque izq.	LT/L2
ATAQUE DERECHO	CLICK DER.	Botón ataque der.	RT/R2
ESQUIVE/PARRY	A,D	Joystick táctil izq.	Joystick izq.
BLOQUEO	ESPACIO	Botón bloqueo	B/X
DASH	ESPACIO	Botón dash	B/X

Por otro lado, los agentes que interactúan con el entorno y el jugador en el juego son:

- Personaje principal: Controlado por el jugador.
- Enemigos:
 - Dummy (Tutorial)
 - Básico (Melee)
 - Bomba (Melee)
 - Heavy (Distancia/Melee)
 - Shooter (Distancia)
 - Boss

ESPECIFICACIONES DE PERSONAJE

Enemigo 1: Básico (Melee)

Descripción

Realiza acciones compuestas de golpes desde cualquier dirección (izquierda / derecha) en distintas formas y combinaciones. No tiene ninguna mecánica adicional además de perseguir al jugador y atacarle cuando esté suficientemente cerca.

Comportamiento:

- El enemigo caminará hasta al jugador sincronizándose con el resto para no ir todos al mismo lado de este, formando una especie de círculo de Kung-Fu (explicado en la sección de algoritmos)
- Cuando este suficientemente cerca, irá turnándose con el resto de los enemigos para atacar al jugador.
- Irá intercalando ataques y defensas en alguna dirección. Si otro enemigo tiene el turno de ataque, mantendrá su pose de defensa.
- Si bloquea al jugador durante su defensa hará un contraataque.
- El enemigo básico que no esté fijado busca la espalda del jugador para atacarle y pillarle desprevenido.
- Si el jugador se aleja del enemigo, este terminará su acción actual, para después devolver su turno de ataque y su posición en el Círculo de Kung-Fu. Tras esto vuelve a ir hacia el jugador.
- Si el enemigo es golpeado por el jugador en una dirección que no está defendiendo, hará una animación de recibir daño y reiniciará su árbol de comportamiento (esta mecánica no se refleja en ningún árbol o máquina de estados, ya que va por encima del sistema de IA).
- Si un enemigo bomba (explicado más adelante) empieza a cargar su explosión cerca de este enemigo, este se defenderá para intentar protegerse de la explosión (la explosión le afecta igualmente, este comportamiento es puramente estético).

Tabla de percepciones

Nombre	Implementación	Acceso
seePlayer	Comprobar si la distancia entre enemigo y jugador es menor que seeDistancia.	Pull
inAttackDistance	Comprobar si la distancia entre enemigo y jugador es menor que attackDistancia.	Pull
TryGetAttackPriority	Comprobar si nadie está atacando ya al jugador.	Pull
isLocked	Comprobar si el enemigo es el personaje fijado por el jugador.	Pull

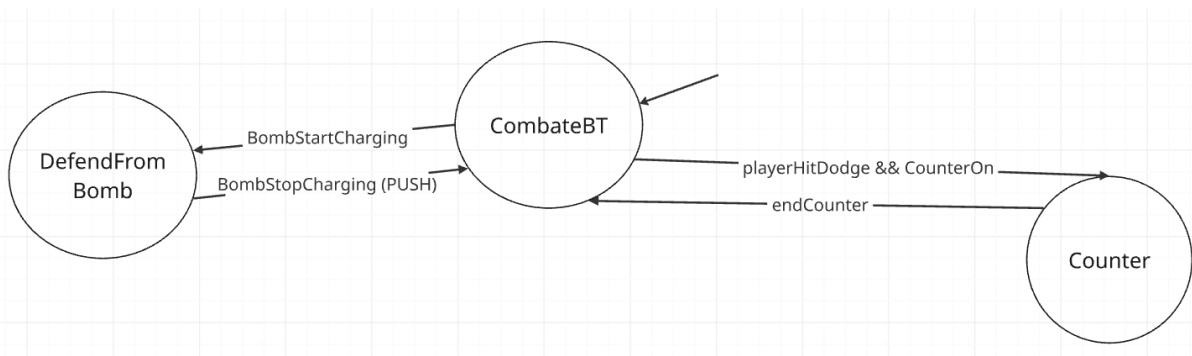
PlayerHitDodge	Recibir evento de que el jugador ha golpeado donde el enemigo se defendía.	Push
counterOn	Variable Interna para saber si el personaje va a hacer counter a los ataques del jugador. Por ejemplo: un enemigo con superarmour no hará ningún counter al jugador.	Pull

Tabla de acciones

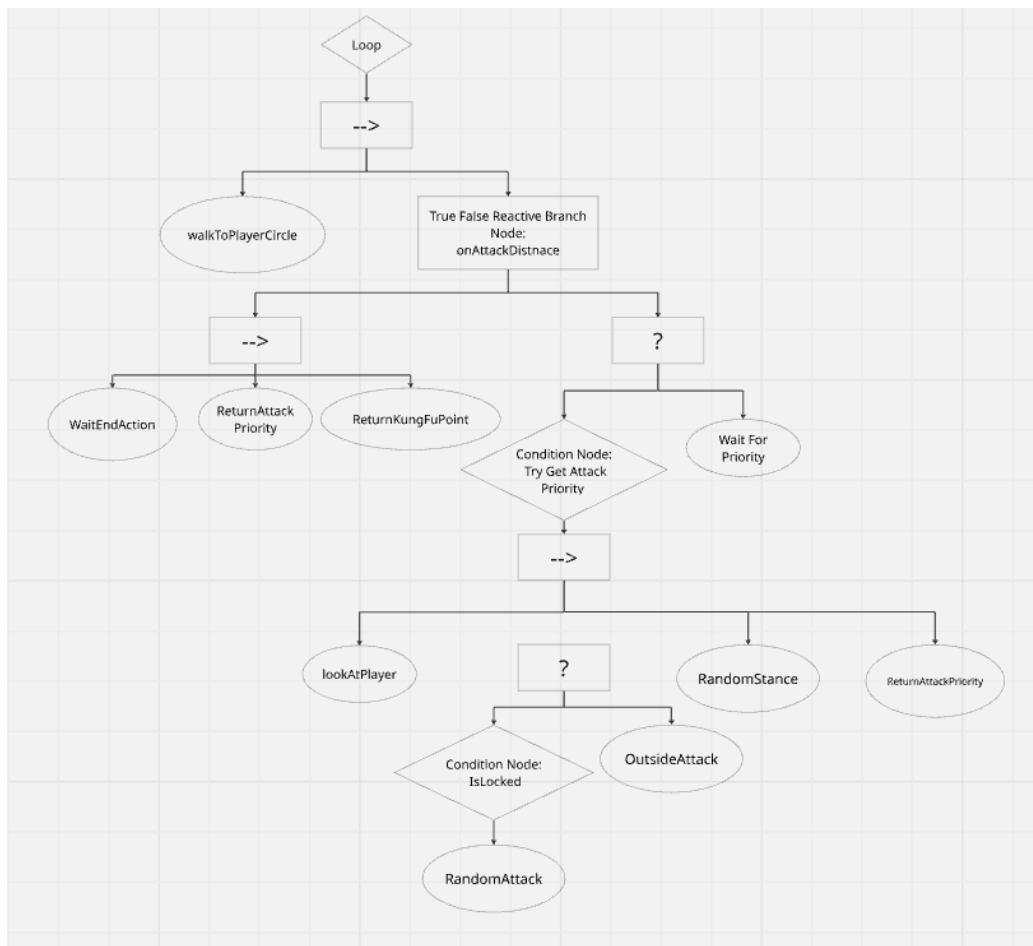
Nombre	Implementación	Efecto
WalkToPlayerCircle	Avanza hacia el jugador, sincronizándose con el resto de enemigos para colocarse en un <i>Círculo de Kung-Fu</i> . Si no hay puntos de Kung-Fu disponibles esta acción mantendrá una animación de Idle hasta que alguno quede libre.	El enemigo se acerca al jugador. Varios enemigos le rodearán.
RandomAttack	Elige aleatoriamente uno de sus ataques.	El enemigo hace una combinación de ataques.
OutsideAttack	Inicia Animación OutsideAttack.	El enemigo (no fijado) ataca el jugador.
WaitEndAction	Espera a que termine la acción actual.	El enemigo termina su animación actual.
DefendFromBomb	Inicia una postura de defensa.	El enemigo intenta protegerse de la explosión de la bomba (sin mucho efecto).
RandomStance	Inicia una animación de Defensa.	El enemigo se defiende en una dirección aleatoria.
Counter	Inicia animación de Counter.	El enemigo hace counter cuando el jugador le golpea donde se estaba bloqueando.
ReturnAttackPriority	Termina su turno de ataque.	El siguiente enemigo ataca al jugador.
ReturnKungFuCircle	Devuelve su posición reservada en el <i>Círculo de Kung-Fu</i> .	Otros enemigos podrán colocarse en esa posición.
WaitForPriority	Espera a que el toque su turno de atacar manteniendo su defensa actual.	El enemigo se defiende durante un tiempo, dejando una oportunidad de ataque.
GetHit	Hace una animación de recibir daño.	El enemigo es visiblemente golpeado.

Diagramas descriptivos

- Nivel 1: FSM Combate (Combat)



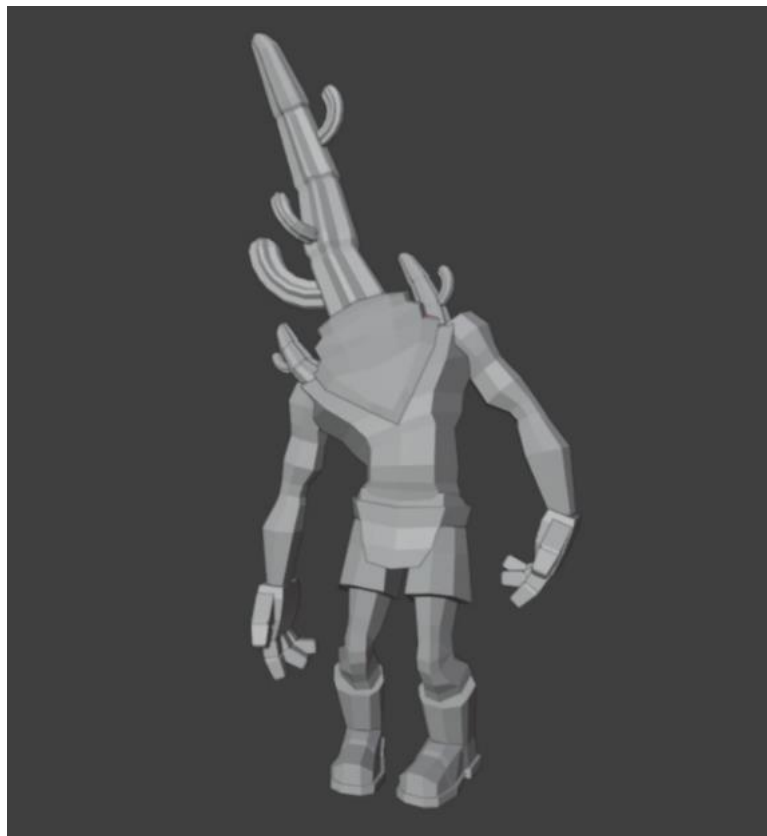
- Nivel 2: Árbol de comportamiento (CombatBT)



Estructuras de datos

Nombre	Significado	Tipo	Unidad
See Distance	Distancia mínima a la que detecta al jugador.	Float	Metros
Attack Distance	Distancia mínima a la que puede atacar al jugador.	Float	Metros
Speed	La velocidad de movimiento del enemigo.	Float	Metros/frame

Modelo del enemigo básico



Modelo del básico

Aclaración: Varios enemigos tienen ciertas acciones y percepciones en comun (se acercan a ti, tienen una serie de ataques aleatorios, piden turno en el Mutex...). Por ello, se han destacado las acciones y percepciones únicas de los siguientes personajes en verde para una mayor legibilidad.

Enemigo 2: Dummy (Tutorial)

Descripción

Se trata del enemigo del tutorial; un cactus que realiza ataques básicos. El jugador se familiarizará con la interfaz de ataque y aprenderá a esquivar golpes de enemigos y contraatacar.

Existen dos versiones: una primera completa que simula un enemigo a melee, la cual permite al enemigo atacar, defenderse, realizarle un parry... También puede ser atacado.

Y una segunda versión que sirve al jugador como aprendizaje para aprender a bloquear ataques externos. Para ello, y mientras el jugador está lockeando al cactus ya mencionado, aparece un segundo cactus que solo puede atacar.

A mitad de tutorial, cuando el *dummy* está a mitad de golpe, se congela la pantalla y aparece un mensaje avisándote que puedes realizar un *parry* esquivando en un momento concreto.

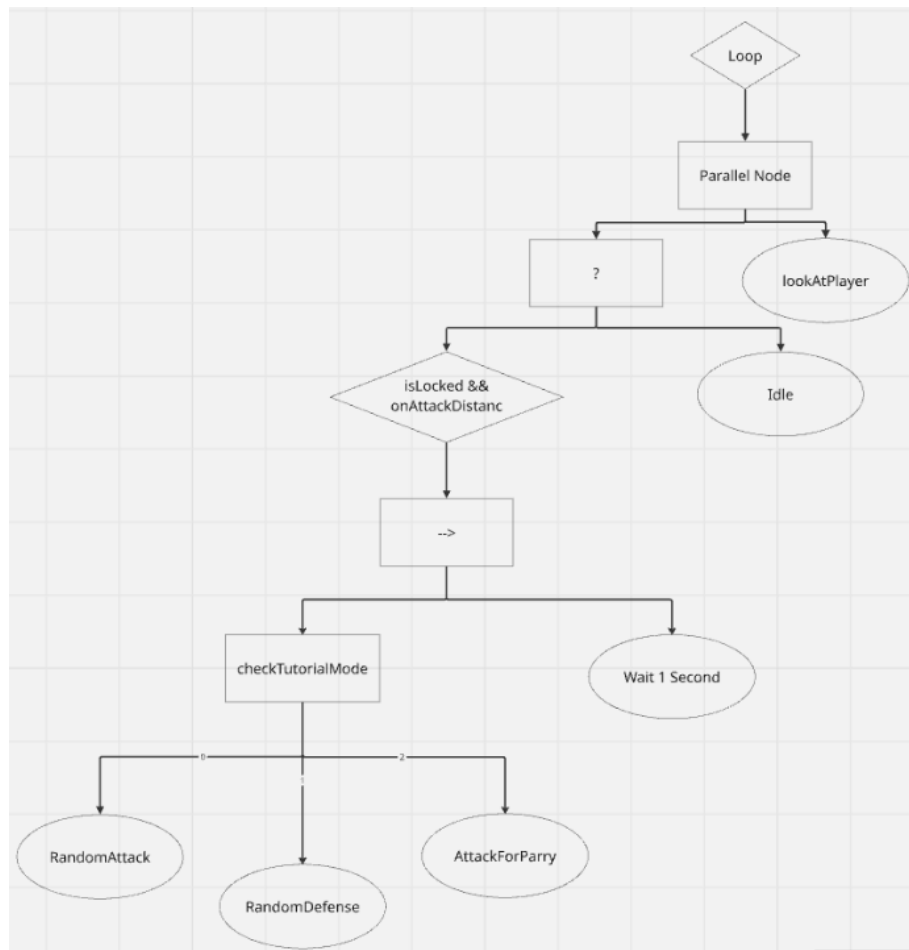
Tabla de percepciones

Nombre	Implementación	Acceso
seePlayer	Comprobar si la distancia entre enemigo y jugador es menor que seeDistancia.	Pull
inAttackDistance	Comprobar si la distancia entre enemigo y jugador es menor que attackDistancia.	Pull
isLocked	Comprobar si el enemigo es el personaje fijado por el jugador.	Pull
checkTutorialMode	Comprobar el estado actual del tutorial.	Pull

Tabla de acciones

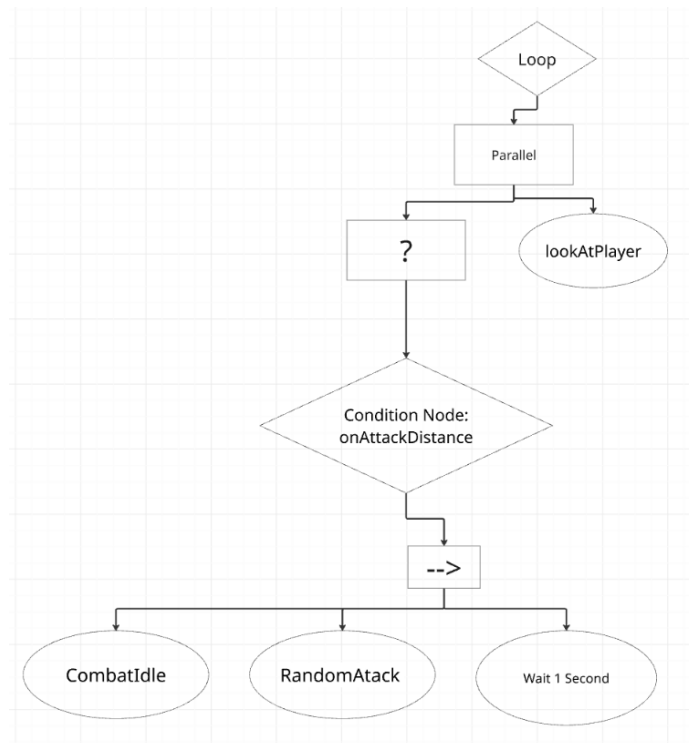
Nombre	Implementación	Efecto
RandomAttack	Inicia Animación de Atacar.	Ataca al jugador.
RandomStance	Inicia una animación de Stance.	El enemigo se defiende en una dirección aleatoria.
AttackForParry	Inicia Animación de AtacarTutorialParry.	Hace un ataque que se parará en el momento donde se debería hacer el Parry para que el jugador aprenda la mecánica.
lookAtPlayer	Cambio su rotación para apuntar al jugador.	El enemigo siempre estará mirando al jugador.
CombatIdle	Inicia animación Idle.	El enemigo realiza un ciclo de su animación de idle.

Diagrama descriptivo



Behaviour Tree del Primer Dummy

Este primer dummy sirve en el tutorial como introducción al combate 1 vs 1 fundamental del juego. Para ello, recoge todas las funcionalidades de combate. Primero, se comprueba si está lockeado y se encuentra a una distancia mínima para atacar. Existen varias acciones que realizará dependiendo del punto del tutorial en el que se encuentre el jugador, pudiendo el cactus realizar un ataque aleatorio, una defensa aleatoria o un ataque para que el jugador haga un *parry*. Luego, esperará un segundo para realizar otra acción.



Behaviour Tree del Segundo Dummy

Este segundo dummy es más simple que el anterior, siendo su única función práctica atacar al jugador mientras este lockea a otro enemigo. Para ello, realiza en bucle el idle del combate, realiza un ataque aleatorio y espera 1 segundo, todo esto mirando al jugador.

Estructuras de datos

Nombre	Significado	Tipo	Unidad
See Distance	Distancia mínima a la que detecta al jugador.	Float	Metros
Attack Distance	Distancia mínima a la que puede atacar al jugador.	Float	Metros

Modelo del enemigo “dummy”



Modelo del dummy

Enemigo 3: Bomba (Melee)

Descripción

Este enemigo se acerca a ti y se inmola en un ataque explosivo. Antes de explotar, cuando el enemigo ya se ha acercado suficiente, se va hinchando (o cargando) durante unos segundos. Durante este lapso, el jugador puede, en lugar de huir de la inminente explosión, atreverse a golpear al enemigo. Esto hace que el enemigo salga volando y explote en el lugar del impacto, pudiendo utilizarse para dañar a otros enemigos.

Utilizar el gancho para atraer a este enemigo provocará que te explote en la cara, salvo que consigas golpearlo a tiempo, haciendo que salga volando como un proyectil.

Los enemigos de tipo Heavy (explicados más adelante) podrán lanzar a los enemigos Bomba y usarlos como proyectil.

El resto de los enemigos tendrán comportamientos menores si una bomba empieza a cargarse cerca de ellos. Estos comportamientos no tienen gran efecto en el gameplay y sirven principalmente para que los enemigos se sientan más vivos.

Tabla de percepciones

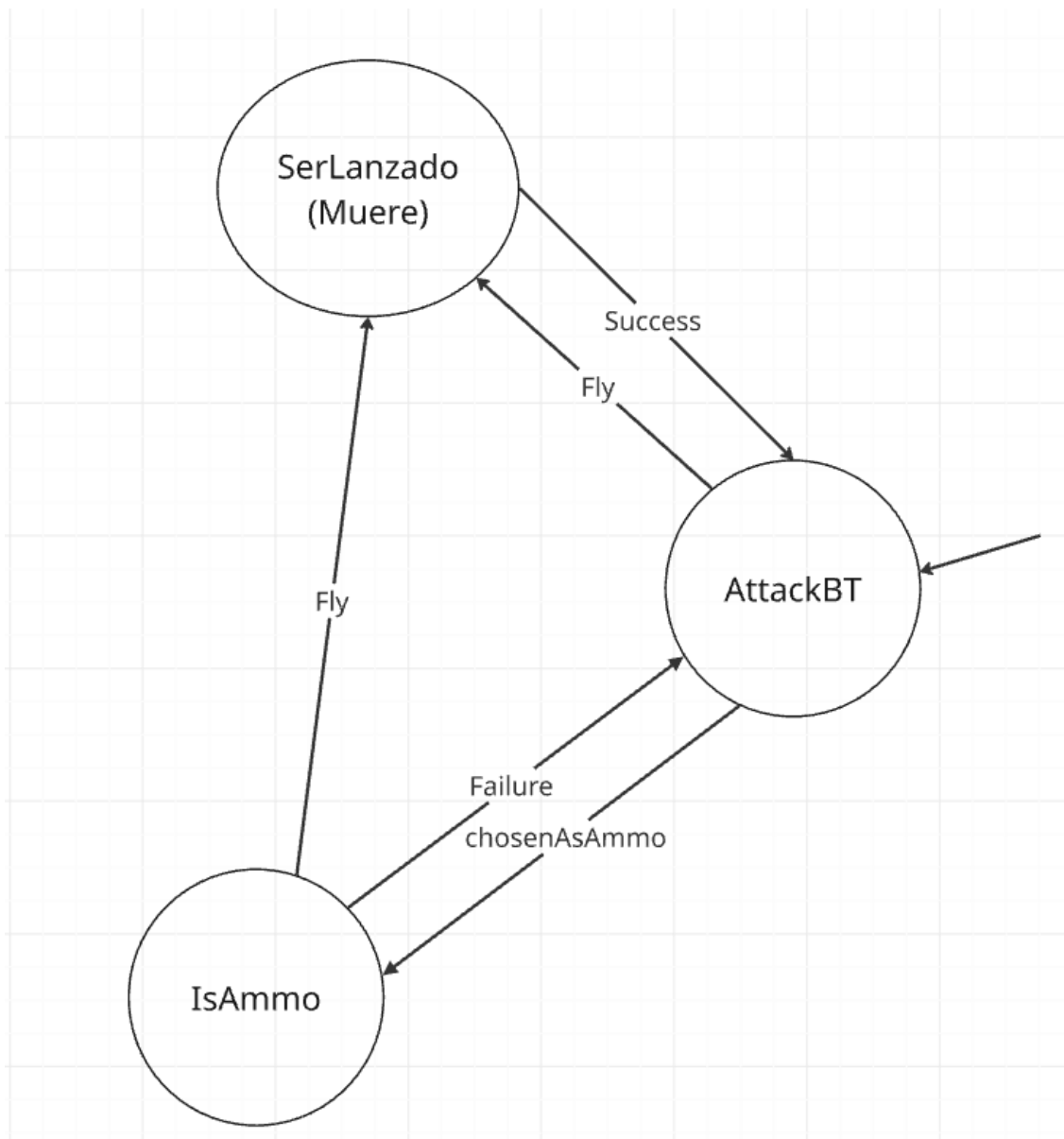
Nombre	Implementación	Acceso
seePlayer	Comprobar si la distancia entre enemigo y jugador es menor que seeDistancia.	Pull
inAttackDistance	Comprobar si la distancia entre enemigo y jugador es menor que attackDistancia.	Pull
charging	Si el enemigo está en su animación de cargar explosión.	Pull
chosenAsAmmo	Evento de que un Heavy lo elija como munición.	Push
Fly	Evento de ser lanzado ya sea por un Heavy o por el jugador.	Push

Tabla de acciones

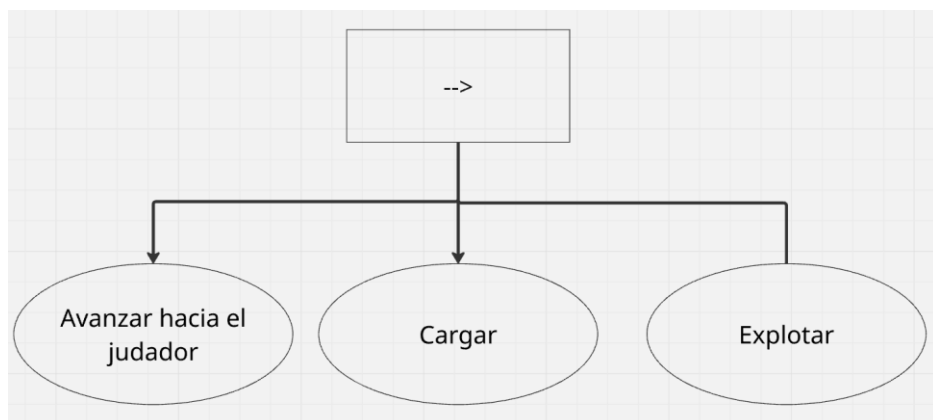
Nombre	Implementación	Efecto
Avanzar	Inicia avance hacia el jugador.	El enemigo se acerca al jugador.
Cargar	Inicia animación de explosión y avisa a enemigos cercanos.	El enemigo empieza a hincharse.
Explotar	Aparece un VFX de explosión y se hace daño a los personajes cercanos.	El enemigo explota al rato haciendo daño al jugador.
SerLanzado	Gana mucha velocidad en una dirección determinada y explota si toca algo.	El enemigo sale volando.
IsAmmo	Añade al Heavy a su objetivo de NavMesh. Si el Heavy muere o es interrumpido devuelve Failure.	El enemigo empieza a andar hacia el Heavy.

Diagramas descriptivos

- Nivel 1: FSM (Combat)



- Nivel 2: BT (AttackBT)



Estructuras de datos

Nombre	Significado	Tipo	Unidad
See Distance	Distancia mínima a la que detecta al jugador.	Float	Metros
Attack Distance	Distancia mínima a la que puede atacar al jugador.	Float	Metros
Speed	La velocidad de movimiento del enemigo.	Float	Metros/frame
Explosion Delay	Cuanto tarda en explotar el personaje.	Float	Segundos
Explosion Radius	El radio de la explosión.	Float	Metros
ChargingWarningDistance	La distancia a la que se les avisa a los enemigos de que va a haber una explosión.	Float	Metros

Modelo del enemigo “bomba”



Modelo del suicida

Enemigo 4: Heavy (Distancia / Melee)

Descripción

Se trata de un enemigo tanque, con más opciones de acción, vida y daño de golpeo. Puede lanzar piedras hacia el jugador. Las piedras se encuentran repartidas en montones por el escenario de manera **limitada**, sirviendo de munición.

- Si se queda sin montones de piedra y sin munición, se acerca a ti y te golpea cuerpo a cuerpo (árbol de combate)
- Este enemigo tiene en cuenta diversos factores, siendo estos su distancia al jugador, su distancia a la piedra más cercana, o si el jugador tiene a otros enemigos cerca. Utiliza estos factores para decidir si debe atacarte a melee o a distancia mediante un Utility System.
- Es más probable que te ataque a distancia si tiene munición cerca o si tienes cerca a enemigos (que te mantienen ocupado). En ese caso el Heavy se acercará a la piedra y cuando llegue la lanzará hacia el jugador.
- Es más probable que te ataque a melee si estas cerca de él, habiendo un límite de distancia para el que ya no podrá atacarte desde lejos sin importar el resto de los factores.
- Si hay algún obstáculo entre el jugador y una bala, esta bala no se tendrá en cuenta a la hora de buscar la bala más cercana.
- La bala más cercana puede cambiar en cualquier momento si hay algún cambio de distancia (el enemigo es agarrado con el gancho) o si el jugador se coloca tras una pared que bloquearía la bala.
- El Heavy tiene superarmor en todos sus ataques y también en su animación de idle, haciendo que solo sea vulnerable al terminar un ataque.

El enemigo puede utilizar de manera alternativa a la munición normal al **enemigo bomba**.

Tabla de percepciones

Nombre	Implementación	Acceso
seePlayer	Comprobar si la distancia entre enemigo y jugador es menor que seeDistancia.	Pull
inAttackDistance	Comprobar si la distancia entre enemigo y jugador es menor que attackDistancia.	Pull
TryGetAttackPriority	Comprobar si nadie está atacando ya al jugador.	Pull

isLocked	Comprobar si el enemigo es el personaje fijado por el jugador.	Pull
_closestAmmo	El objeto lanzable válido más cercano.	Pull
_currentAmmo	El objeto lanzable al que se está yendo actualmente.	Pull
DistanciaJugador	Distancia al jugador.	Pull
DistanciaCactusCercano	Distancia a closestAmmo.	Pull
Enemigos Cercanos a Jugador	El número de enemigos cerca del jugador.	Pull

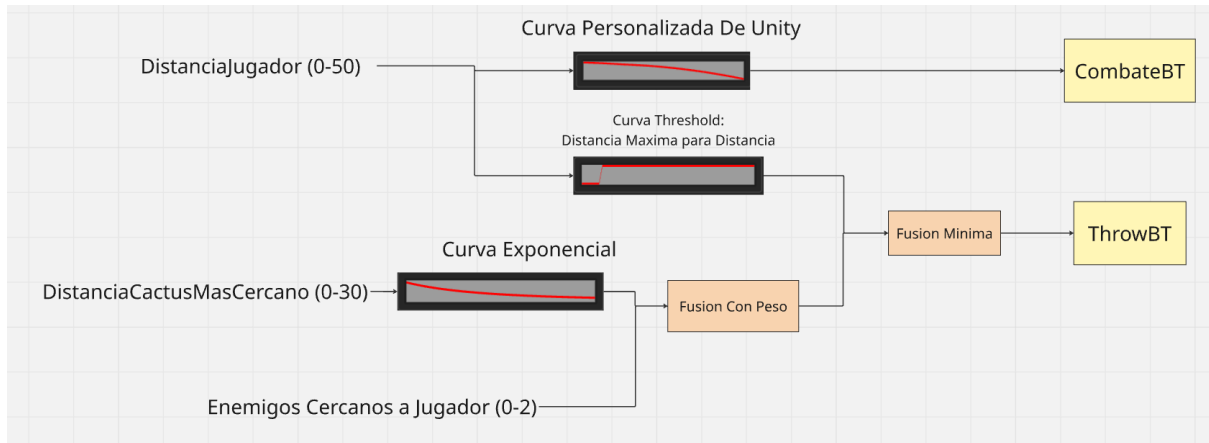
Tabla de acciones

Nombre	Implementación	Efecto
WalkToPlayerCircle	Avanza hacia el jugador, sincronizándose con el resto de enemigos para colocarse en un <i>Círculo de Kung-Fu</i> .	El enemigo se acerca al jugador. Varios enemigos le rodearán.
RandomAttack	Elige aleatoriamente una de sus animaciones de ataque.	El enemigo hace una combinación de ataques.
WaitEndAction	Espera a que termine la acción actual.	El enemigo termina su animación actual.
OutsideAttack	Inicia animación OutsideAttack.	El enemigo (no fijado) ataca el jugador.
CombatIdle	Inicia animación Idle.	El enemigo realiza un ciclo de su animación de idle.
Counter	Inicia animación de Counter.	El enemigo hace counter cuando el jugador le golpea donde se estaba bloqueando.
Throw	Hace una animación de lanzar al enemigo y llama a la acción Thrown() del enemigo (si la hay).	El Heavy lanza al enemigo o a la piedra.
RunToAmmo	Añade la posición de closestAmmo a su objetivo de NavMesh. Es posible que el objetivo cambie a lo largo del camino.	El Heavy anda hacia el objeto lanzable más cercano.
ReturnAttackPriority	Termina su turno de ataque.	El siguiente enemigo ataca al jugador.
ReturnKungFuCircle	Devuelve su posición reservada en el <i>Círculo de Kung-Fu</i> .	Otros enemigos podrán colocarse en esa posición.
WaitForPriority	Espera a que el toque su turno de atacar manteniendo su defensa actual.	El enemigo se defiende durante un tiempo, dejando una oportunidad de ataque.

GetHit	Hace una animación de recibir daño.	El enemigo es visiblemente golpeado.
--------	-------------------------------------	--------------------------------------

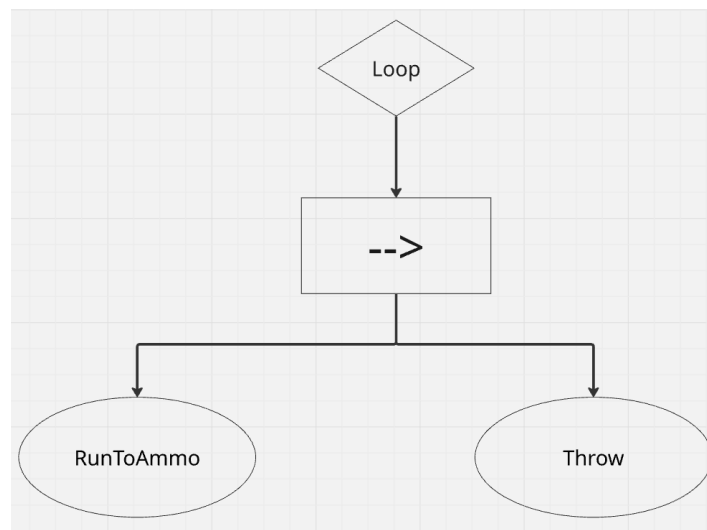
Diagramas descriptivos

- Nivel 1: Utility System

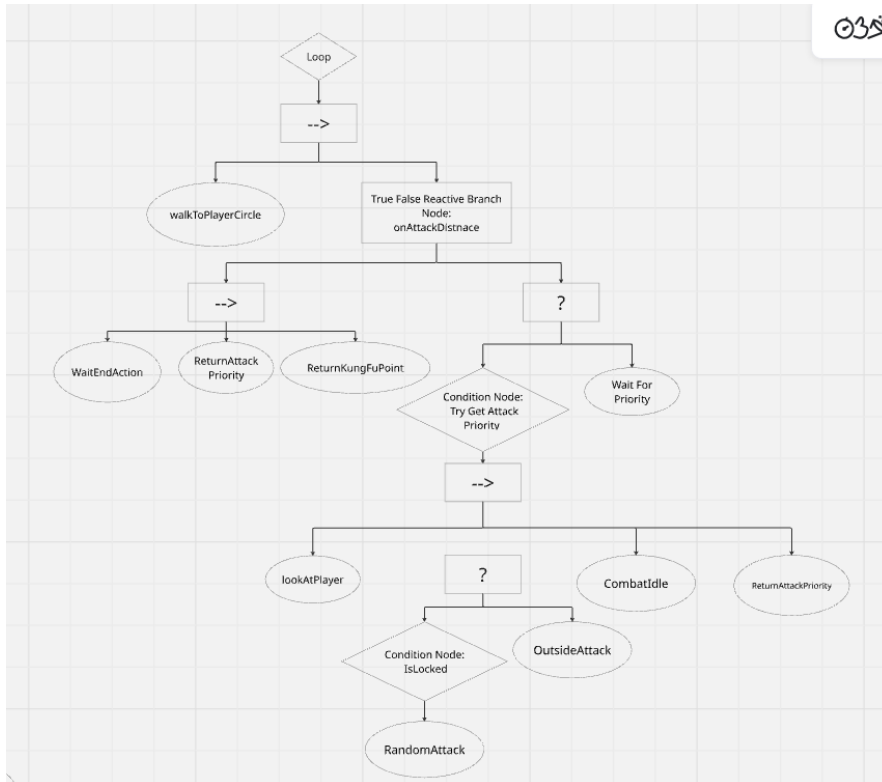


Este enemigo es el único que utiliza un Utility System combinado con árboles de comportamiento, lo que lo hace único también del resto.

- Nivel 2: Árbol de Comportamiento (ThrowBT)



- Nivel 2: Árbol de comportamiento (CombateBT) (Prácticamente igual a Basic)



Este árbol tiene la misma estructura que el árbol de los enemigos básicos, con las mismas funcionalidades. Sin embargo, cuando tiene prioridad y ha atacado, realizará un idle (con superarmor) en vez de las defensas direccionales de los Básicos.

Estructuras de datos

Nombre	Significado	Tipo	Unidad
See Distance	Distancia mínima a la que detecta al jugador.	Float	Metros
Attack Distance	Distancia mínima a la que puede atacar al jugador.	Float	Metros
Speed	La velocidad de movimiento del enemigo.	Float	Metros/frame
Throw Ammo Distance	Distancia máxima y mínima a la que puede lanzar piedras y/o enemigos al jugador.	Float	Metros
Min Distance Threshold	La distancia mínima para poder atacar a distancia.	Float	Metros

Modelo del enemigo “heavy”



Concept del heavy



Modelo del heavy

Enemigo 5: Shooter (Distancia)

Descripción

Este enemigo no busca golpearte a corta distancia, si no mantenerse lejos y dispararte con su revolver. Si lo acorralas y le haces combatir, buscará cualquier oportunidad para escaparse y volver a ponerse lejos de ti.

El mapa tiene varias “coberturas”. Este enemigo se encuentra apostado en estos puntos donde se recoloca para cubrirse siempre del jugador. Si ninguno de estos puntos le sirve para cubrirse, busca otra cobertura.

Al acercarte a él, y fijarlo, entras en modo combate (enemigo igual que los demás). Sus mecánicas de combate son las más básicas: no tiene iniciativa ofensiva, solo se defiende. Cuenta además con un contraataque que puede aturdir al jugador, aprovechando el *Shooter* esta oportunidad para huir una vez más hacia alguna cobertura. Si eres rápido, puedes engancharle antes de que consiga cubrirse y rematarlo. Además, si en cualquier momento el jugador deja de fijar al *Shooter* (para atacar a otro enemigo o recolocarse), este saldrá corriendo a una de las coberturas.

Si un enemigo bomba empieza a cargar su explosión cerca del Shooter, este empezara a huir de él.

Tabla de percepciones

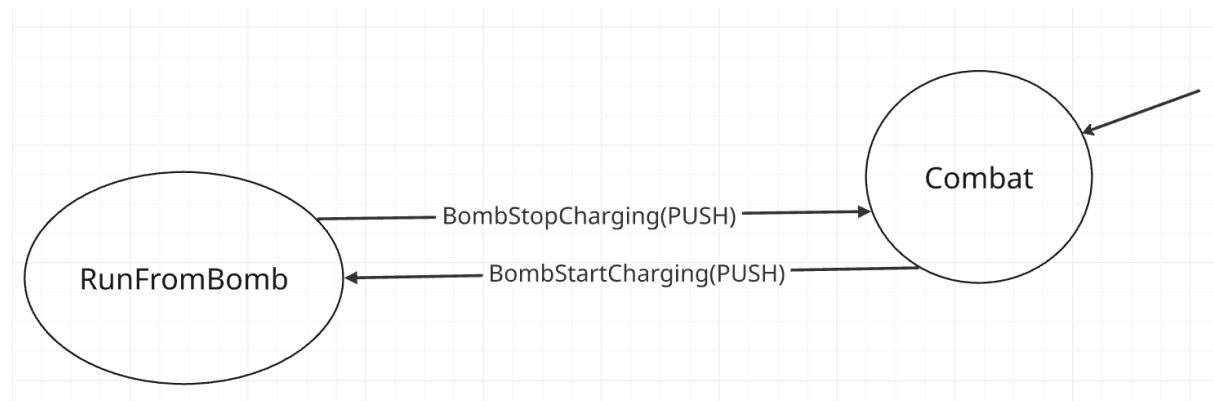
Nombre	Implementación	Acceso
seePlayer	Comprobar si la distancia entre enemigo y jugador es menor que seeDistancia.	Pull
inAttackDistance	Comprobar si la distancia entre enemigo y jugador es menor que attackDistancia.	Pull
TryGetAttackPriority	Comprobar si nadie está atacando ya al jugador.	Pull
isLocked	Comprobar si el enemigo es el personaje fijado por el jugador.	Pull
PlayerHitDodge	Recibir evento de que el jugador ha golpeado donde el enemigo se defendía.	Push
counterOn	Variable Interna para saber si el personaje va a hacer counter a los ataques del jugador.	Pull
findClosestCover	Encuentra la cobertura más cercana.	Pull
isOnCover	Comprueba si está en una cobertura.	Pull
EndQueue	Recibir evento de que le toca el Mutex de atacar.	Push
BombStartCharging	Una bomba cercana empieza a cargarse para explotar.	Push
BombStopCharging	La bomba que empezó a cargar para, ya sea porque ya ha explotado o porque ha sido interrumpida.	Push

Tabla de acciones

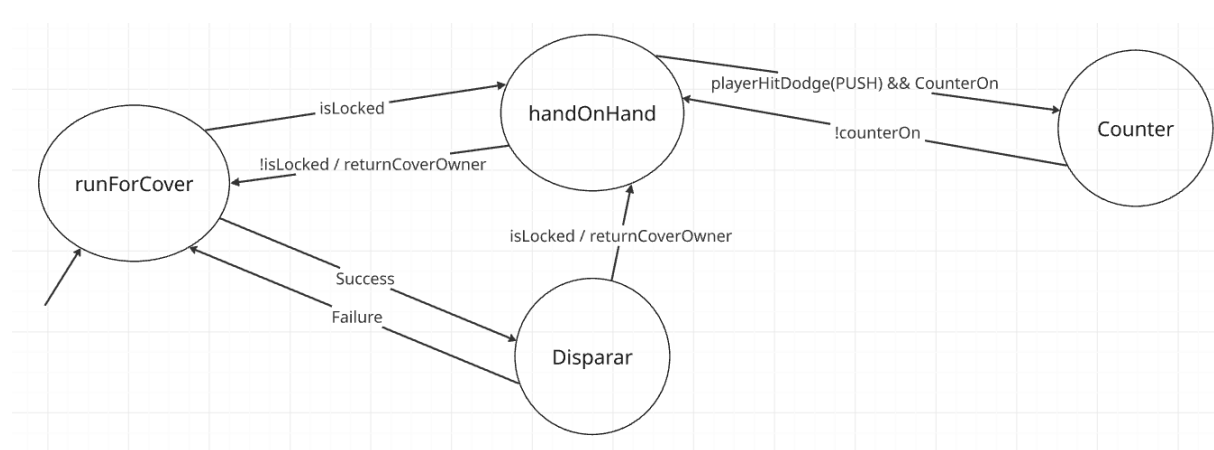
Nombre	Implementación	Efecto
RunForcover Action	Calcula la cobertura segura más cercana y añade su posición como objetivo de NavMesh.	El enemigo se corre hacia la cobertura segura más cercana.
Disparar	Crea una bala que va hacia el jugador.	Dispara una bala hacia el jugador.
Wait For Priority	Inicia animación Idle mientras espera que le toque el Mutex. Se termina cuando recibe la percepción PUSH EndQueue.	El enemigo realiza un ciclo de su animación de idle en cobertura.
handOnHand	Inicia animación de Idle Reactivo y activa counterOn.	Si el jugador golpea una de las direcciones donde se defendía, realiza un counter.
Counter	Inicia animación de <i>Counter</i> .	El enemigo hace <i>counter</i> cuando el jugador le golpea donde se estaba bloqueando. (El counter hace que el protagonista se <i>stunee</i> unos segundos permitiendo que el enemigo huya).
lookAtPlayer	Cambio su rotación para apuntar al jugador.	El enemigo mirará al jugador mientras ataca.
Random Stance	Inicia una animación de defensa aleatoria.	El enemigo se defiende por una dirección aleatoria.
Bark	Inicia animación Bark.	El enemigo reacciona al ver al jugador o al ver que ya no está seguro.
RunFromBomb	Calcula una dirección alejada del enemigo Bomba y la añade como objetivo en su NavMesh.	El enemigo sale corriendo del bomba.

Diagrama descriptivo

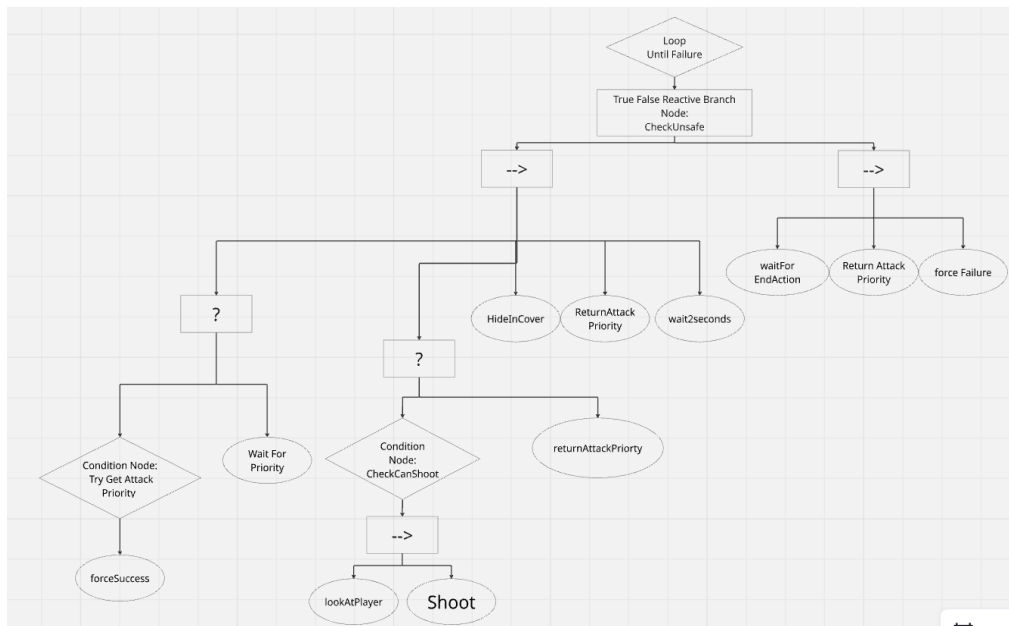
Nivel 1: FSM



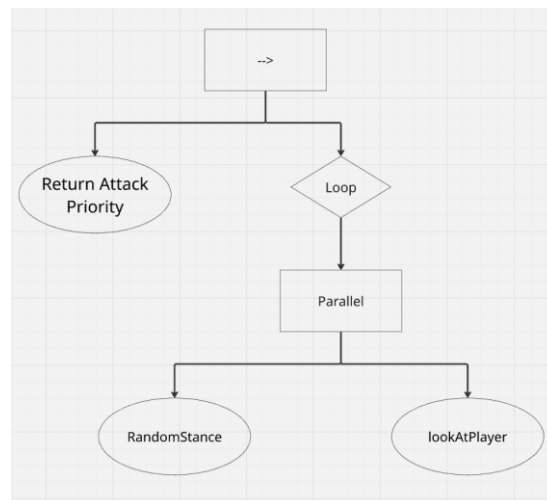
Nivel 2: FSM (Combat)



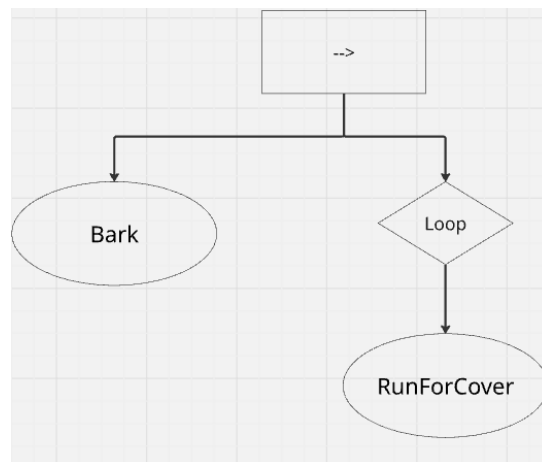
Nivel 2: BT (Disparar)



Nivel 3: BT (handOnHand)



Nivel 3: BT (RunForCover)



Para buscar una nueva cobertura, el enemigo realiza un bark (animación de grito) que le dota de inteligencia dándole una apariencia de asustadizo mientras corre hacia el nuevo punto donde disparará.

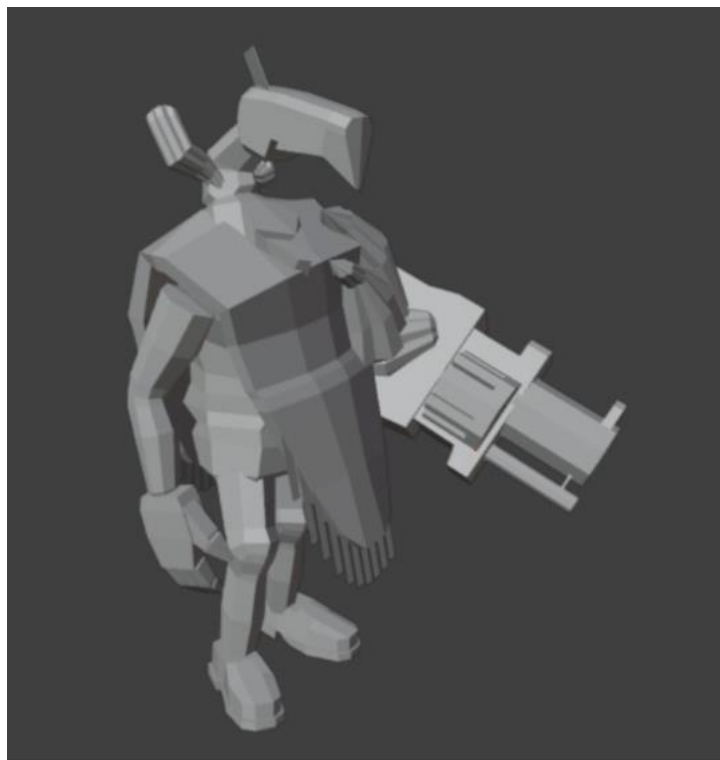
Estructuras de datos

Nombre	Significado	Tipo	Unidad
See Distance	Distancia mínima a la que detecta al jugador.	Float	Metros
Attack Distance	Distancia mínima a la que puede atacar al jugador.	Float	Metros
Speed	La velocidad de movimiento del enemigo.	Float	Metros/frame
Shoot Distance	Distancia máxima y mínima a la que puede disparar jugador.	Float	Metros

Modelo del enemigo “shooter”



Concept de shooter



Modelo del shooter

Enemigo 6: Boss

Descripción

Consiste en el enemigo final, el cual posee una carga sentimental y narrativa inmensa. Se trata de un enemigo mixto que, al igual que el heavy, combina combate a melee y a distancia, aunque de un modo más particular al tratarse de la batalla final.

Consta de varias fases para dar dinamismo al combate y aumentar su dificultad, siendo la pelea 1 vs 1 más complicada. En primer lugar, el enemigo empieza su primera fase cuerpo a cuerpo, con un sistema de ataques aleatorios y combos.

Pasado un tiempo, el enemigo cambia a su segunda fase, donde salta a una roca y dispara desde una distancia prudente al personaje, desde donde no puede ser alcanzado. El personaje deberá esquivar estas balas hasta que el enemigo realice un grito que le deja vulnerable y para finalizar esta fase deberá ser atraído por el gancho.

La última fase consiste en una variación de la primera fase (melee), pero añadiendo un extra de dificultad donde nacen cactus del suelo que realizan una animación de golpeo y dañan al personaje. Estos mini-enemigos (que actúan a modo de minions del boss) spawnen y despawnean muy rápido, dando un tono de frenesí al combate final.

A nivel de IA, el comportamiento mas destacable del Boss es un ataque (Learn) que aprende del jugador. Se trata de un ataque esquivable por ambas direcciones que va guardando hacia donde esquiva el jugador.

En la tercera fase, el Boss usara los datos recogidos para hacer un ataque que empieza igual que Learn, pero lo continua por un ataque hacia un lado, el lado elegido cada vez depende de cuantas veces esquivo a cada lado el jugador.

Dicho de otra forma, un jugador que esquive mucho por la derecha será atacado por la derecha cuando menos se lo espere. Igualmente, el Boss tiene un Bark, en forma de un brillo en el ojo, que indica al jugador donde tendrá que esquivar, esto se ha hecho para evitar que el ataque sea injusto.

Tabla de percepciones

Nombre	Implementación	Acceso
seePlayer	Comprobar si la distancia entre enemigo y jugador es menor que seeDistancia.	Pull
inAttackDistance	Comprobar si la distancia entre enemigo y jugador es menor que attackDistancia.	Pull

TryGetAttackPriority	Comprobar si nadie está atacando ya al jugador.	Pull
isLocked	Comprobar si el enemigo es el personaje fijado por el jugador.	Pull
bossMode	El modo del boss, cambia cuando el jugador supera una fase. Puede ser Distance o Melee.	Pull
learningDodgesOn	Si el boss tiene activado el ataque que aprende del jugador. En caso contrario elegirá ciertos ataques dependiendo de las acciones pasadas del jugador.	Pull
LeftDodges/RightDodges	El número de esquives a cada lado que el jugador ha hecho frente al ataque Learn.	Pull
Choose Best Direction	Devuelve 0 o 1 de manera aleatoria utilizando LeftDodges/RightDodges como probabilidades.	Pull

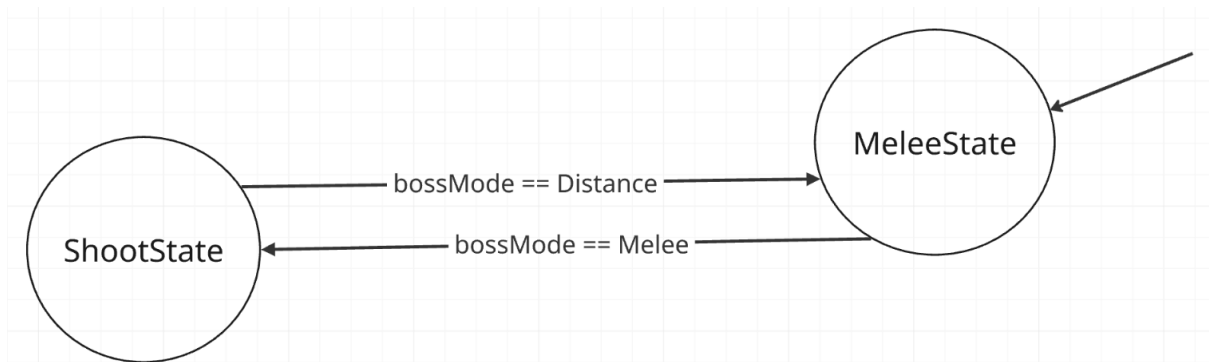
Tabla de acciones

Nombre	Implementación	Efecto
WalkToPlayerCircle	Avanza hacia el jugador, sincronizándose con el resto de enemigos para colocarse en un <i>Círculo de Kung-Fu</i> . Si no hay puntos de Kung-Fu disponibles esta acción mantendrá una animación de Idle hasta que alguno quede libre.	El enemigo se acerca al jugador. Varios enemigos le rodearán.
lookAtPlayer	Cambio su rotación para apuntar al jugador.	El enemigo mirará al jugador mientras ataca.
Shoot	Crea una bala que va hacia el jugador.	Dispara una bala hacia el jugador.
Reload	Inicia animación de Reload.	El enemigo recarga el arma antes de empezar la fase de disparo
CombatIdle	Inicia animación Idle.	El enemigo realiza un ciclo de su animación de idle.
Scream	Inicia animación de grito.	El enemigo grita, dejándole vulnerable al gancho
RandomAttack	Elige aleatoriamente uno de sus ataques.	El enemigo hace una combinación de ataques.

RandomAttack+Learn	Elige aleatoriamente uno de sus ataques. Una de las opciones es el ataque que guarda la dirección a la que esquiva el jugador.	El enemigo hace un ataque aleatorio.
AttackLeft/ AttackRight	Realiza un ataque central seguido de otro por la izquierda o derecha respectivamente.	El enemigo hace un ataque teniendo en cuenta donde suele esquivar el jugador.
ReturnAttackPriority	Termina su turno de ataque.	El siguiente enemigo ataca al jugador.
ReturnKungFuCircle	Devuelve su posición reservada en el Círculo de Kung-Fu.	Otros enemigos podrán colocarse en esa posición.
WaitEndAction	Espera a que termine la acción actual.	El enemigo termina su animación actual.
JumpToRock	Hace una animación de salto y se mueve hasta la piedra.	Hace un salto y cae en una piedra desde donde disparará al jugador.

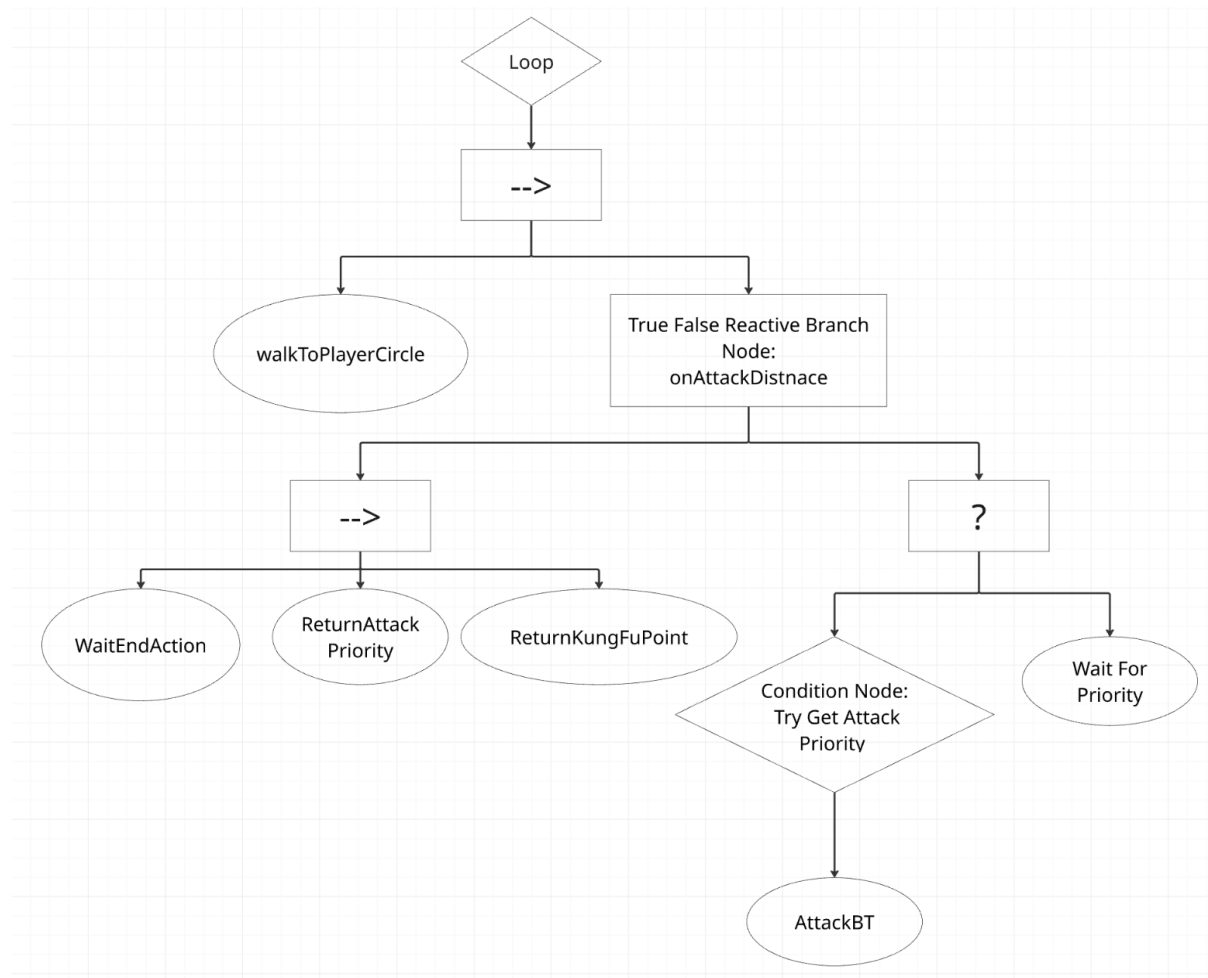
Diagrama descriptivo

Nivel 1: FSM

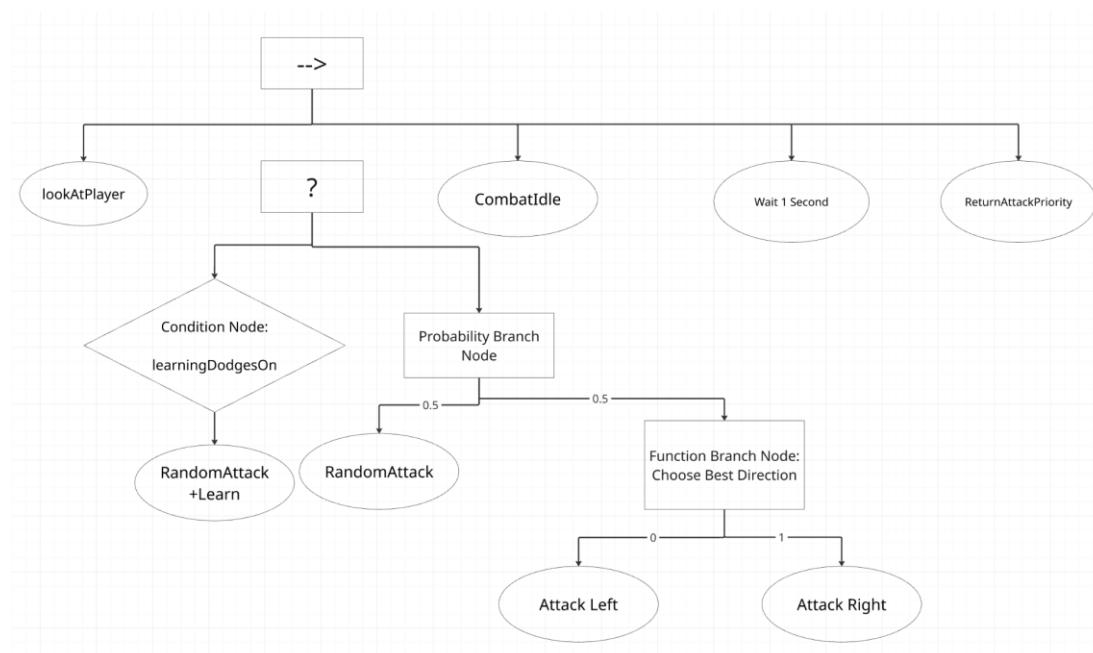


Aunque todo el comportamiento de Melee ocurre en un solo BT, se ha dividido en dos en la memoria para mejor claridad.

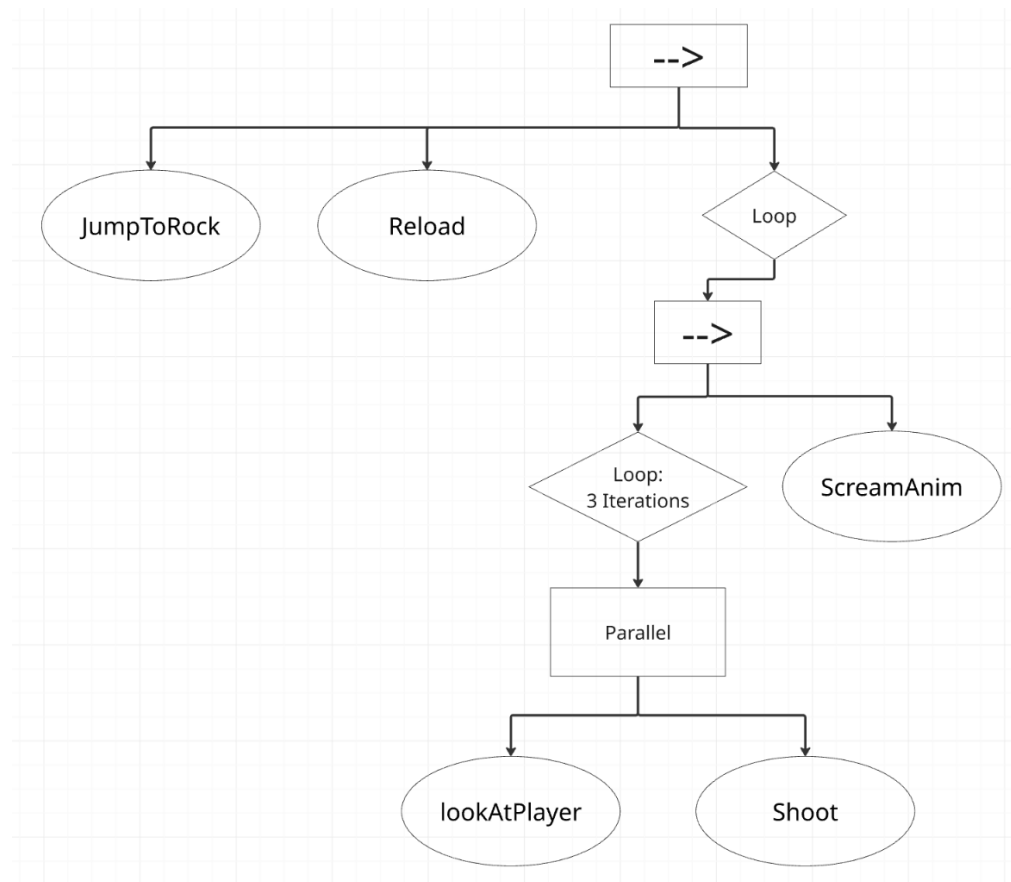
Nivel 2: MeleeStateBT



Nivel3: AttackBT



Nivel 2: ShootBT



Estructuras de datos

Nombre	Significado	Tipo	Unidad
See Distance	Distancia mínima a la que detecta al jugador	Float	Metros
Attack Distance	Distancia mínima a la que puede atacar al jugador	Float	Metros
Speed	La velocidad de movimiento del enemigo	Float	Metros/frame
Shoot Distance	Distancia máxima y mínima a la que puede disparar jugador	Float	Metros
Cactus Attack Time	Cada cuanto tiempo se crea un cactus	Int	Segundos

Modelo del enemigo “boss”



Concept de boss



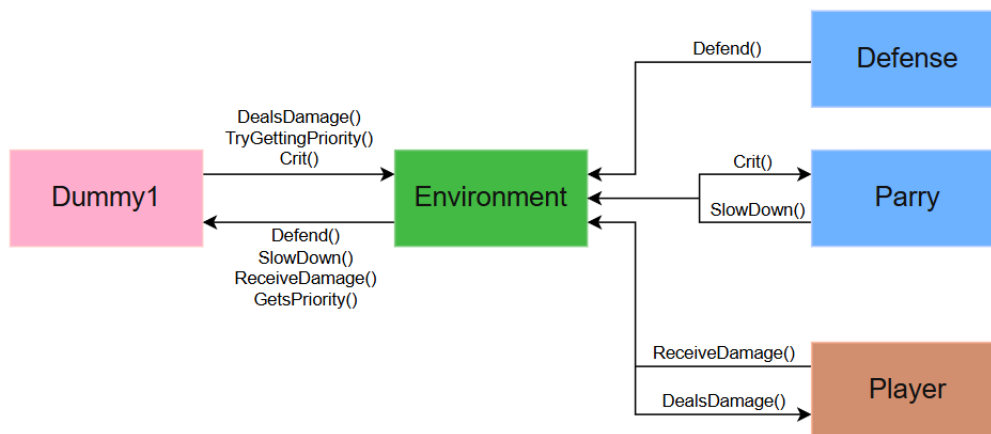
Modelo del boss

INTERACCIONES PERSONAJES – ESCENARIO

Interacción Dummy – Escenario:

La interacción del enemigo de tutorial (dummy) sirve para explicar la interacción básica de los enemigos con el mundo. Este enemigo recibe información del daño efectuado sobre él y envía como información al mundo el daño efectuado contra nuestro personaje. Es capaz de realizar defensas (derecha o izquierda), en las que evitará cualquier daño que provenga de esa dirección. Si el jugador realiza un esquite en una ventana de frames exacta, recibirá información de que se ralentiza el mundo (y su movimiento) y envía que recibirá un incremento de daño en caso de ser alcanzado por un golpe).

Esta estructura base la comparten todos los demás enemigos, sobre la que irán añadiendo funcionalidades.



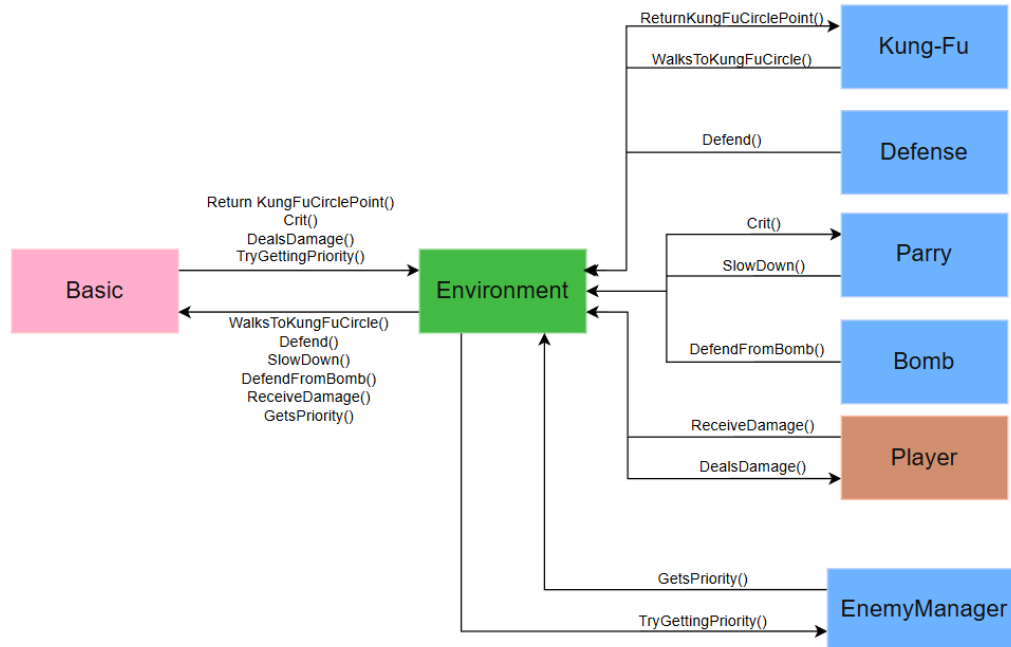
Esta segunda versión del dummy consiste en un enemigo extra que te enseña a defenderte de ataques recibidos por un enemigo externo (no lockeado). Es por ello que su única interacción es atacar al jugador.



Interacción Básico - Escenario:

Este enemigo básico comparte la estructura del dummy, añadiendo como nueva interacción el EnemyManager, del que recibe y envía información de su prioridad de ataque (ya que dos enemigos no pueden atacar a la vez sobre el personaje).

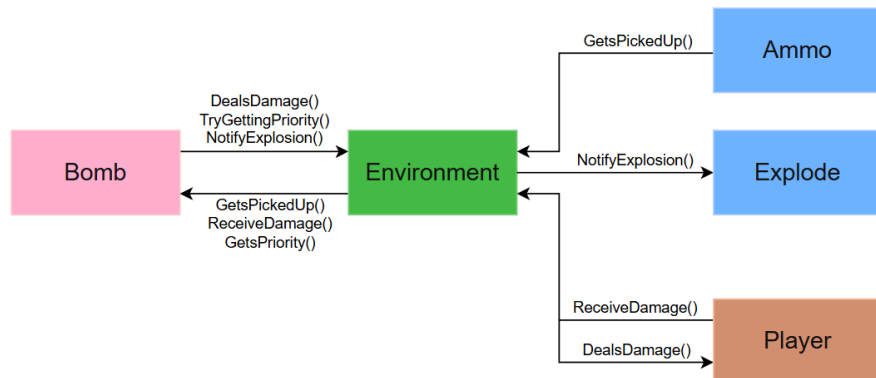
Se han añadido las funcionalidades de defenderse de un enemigo bomba (información que recibe del mundo) y del círculo kung-fu, del que recibe información sobre donde posicionarse y envía el punto de posición.



Interacción Bomba – Escenario:

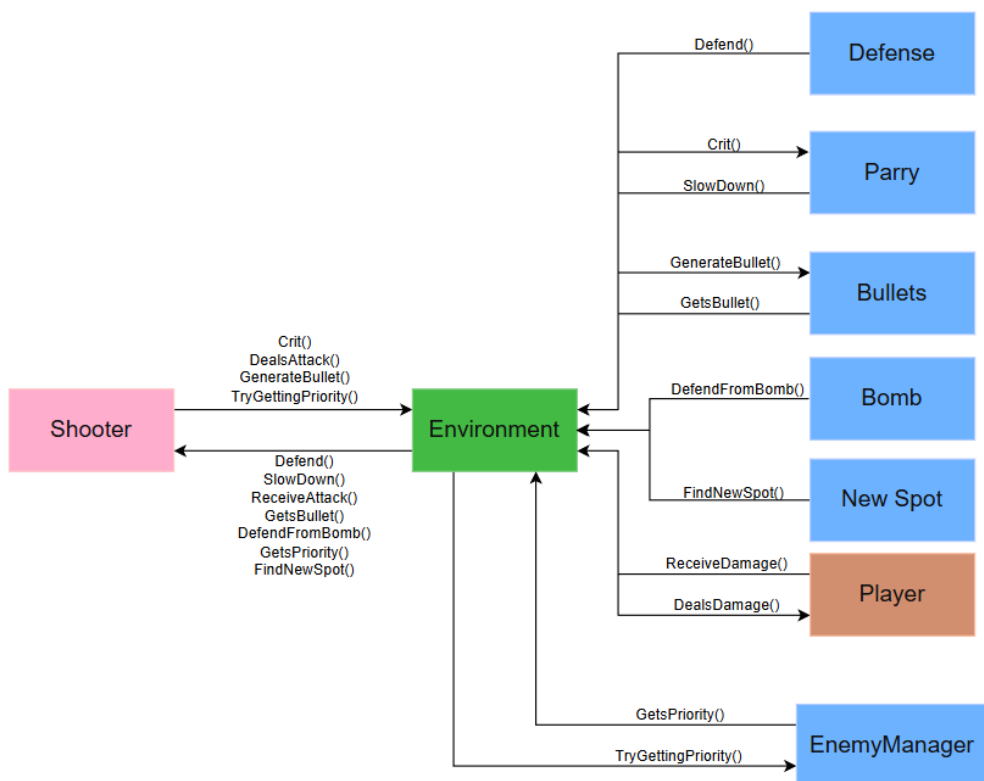
La funcionalidad del enemigo “bomba” es su posibilidad de ser utilizado como arma arrojada por el enemigo “heavy”. Para ello, manda información al mundo de si es

recogido. Puede asestar daño tanto al jugador como al resto de enemigos que se encuentren a su alrededor, notificando que va a explotar para que reaccionen.



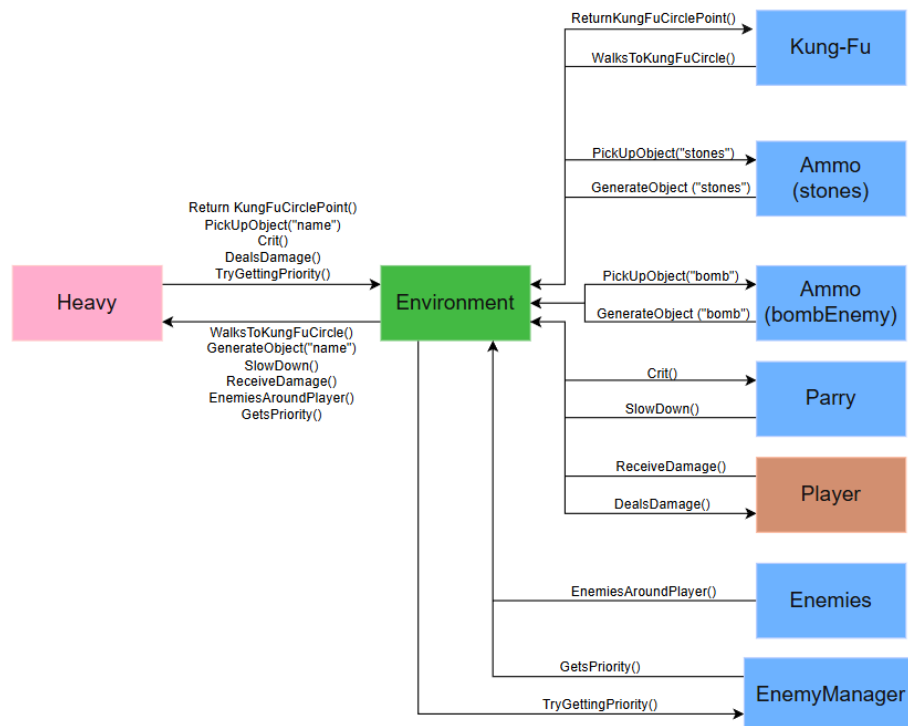
Interacción Shooter – Escenario:

Este enemigo tiene nuevas interacciones con el mundo, enviando información como la generación de balas para disparar al personaje y recibiendo como información la bala generada y la necesidad de buscar un nuevo punto de cobertura.



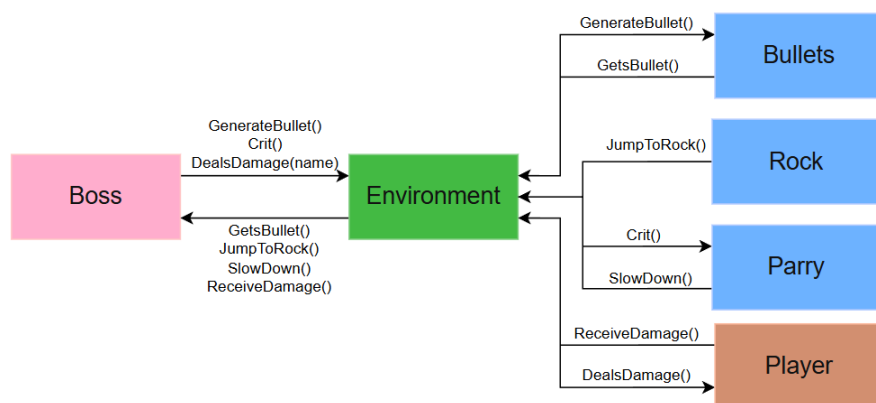
Interacción Heavy – Escenario:

El enemigo heavy, al igual que los básicos, recibe información sobre donde posicionarse y envía el punto de posición. Puede recoger objetos y lanzarlos (piedras y bombas), recibiendo del mundo la generación de estos y enviando información de recogerlos.



Interacción Boss – Escenario:

El boss final envía información al mundo como la generación de balas para disparar y el ataque y daño que asesta al jugador. Por otra parte, recibe información del mundo como las balas generadas para disparar, el daño recibido y la orden de saltar a la roca.



DESCRIPCIÓN DE ALGORITMOS

MUTEX DE ATAQUE

Para que los enemigos se turnen a la hora de atacar, se ha creado un sistema similar a un mutex donde solo el primer enemigo que llega puede atacar y no se permitirá que otros enemigos ataquen hasta que el primero devuelva el mutex.

Para evitar peticiones innecesarias al mutex, este cuenta con una cola de Personajes que quieren atacar y en cuanto quede libre el mutex, se otorgará permiso al primero de la cola.

Por este motivo, todos los objetos que forman parte de un mutex deben implementar la interfaz IMutex que incluye la función `givePermission` que se llama cuando alguien de la cola consigue permiso.

FUNCIONAMIENTO DE COBERTURAS.

Como ya se ha comentado anteriormente, este enemigo se oculta del jugador detrás de determinadas coberturas, colocadas estratégicamente a lo largo y ancho del mapa. Es desde estas coberturas desde las que dispara al personaje.

Cada cobertura tiene uno o varios puntos donde un enemigo se podría esconder.

Para recalcular a qué cobertura nueva debe desplazarse, se lanza un *Raycast* del jugador a cada uno de estos puntos (que se encuentran a ras del suelo) para determinar si es un lugar seguro: si alguno de los puntos detecta algún objeto entre la cobertura y el jugador se determina que el punto de esa cobertura es seguro.

Esta “seguridad” se calcula lanzando el rayo a ras del suelo, ya que los enemigos agacharán tras la cobertura para ponerse a salvo y protegerse totalmente.

Por otra parte, se lanza otro *Raycast* desde una posición algo más elevada que los puntos de cobertura (desde donde se dispararía la bala cuando se ponga de pie) para calcular si es posible disparar al jugador desde esa cobertura.

Usando estos dos *Raycasts*, podemos asegurar que los puntos sean seguros, pero sin que los Shooters intenten disparar a través de una pared, o que si el jugador se pone detrás de una cobertura baja parezca que no se le puede disparar.

Un sistema similar de Raycasts se utiliza para comprobar si existe una trayectoria directa entre el jugador y la munición del Heavy.

CÍRCULO DE KUNG-FU

Para que los enemigos rodearan al jugador sin potencialmente apelotonarse todos en el mismo punto, se creó un sistema que coloca a los enemigos a una distancia fija del jugador y entre ellos de manera similar a otros juegos como Assassin Creed.

El funcionamiento es el siguiente, hay 4 objetos hijos del jugador que le siguen por todo el nivel, estos objetos no rotan cuando lo hace el jugador por lo que se mantiene siempre en la misma posición relativa al jugador, pero sin importar la dirección de su movimiento o de la cámara.

Cuando un enemigo quiere ir hacia el jugador llama a la clase EnemyManager que le proporciona uno de los puntos. Se ha creado la clase OwnerableTransform que asocia un objeto del nivel con una instancia de la clase Enemy, de esta manera nos aseguramos de que varios enemigos no puedan ir al mismo punto.

EnemyManager asigna a cada enemigo el punto libre más cercano al enemigo, evitando que este haga un comportamiento extraño andando más de lo que debería.

Además, el enemigo comprobará periódicamente si hay algún punto libre más cercano a él, para evitar casos donde el jugador salga corriendo, haciendo que el punto objetivo original pare de ser el más cercano.

Un problema que hubo inicialmente es que los cálculos del kung-fu circle se iban a estar calculando continuamente y daba problemas de rendimiento en la versión de web debido al alto número de enemigos en los niveles finales.

Se decidió limitar el número de puntos del círculo a 4 (lo cual no daba problemas ya que en la mayoría de los niveles no hay muchos enemigos Melees) y calcular el punto más cercano comparando la posición x y z de los enemigos y los puntos. Esto era posible ya que los puntos no rotan con el jugador por lo que su posición siempre es la misma con respecto a los ejes de posición. De esta manera se evita tener que hacer cálculos de distancia (mucho más costosos de rendimiento).

ACTUALIZACIÓN DE NAVMESH:

Un problema del enemigo a distancia era que en algunas zonas tomaba caminos haciaa coberturas que pasaban a través del jugador, haciéndolo parecer bastante poco inteligente. Esto era espealmente grave en el nivel uno ya que había una zona circular donde era aparente que si el enemigo se daba la vuelta podría llegar a la cobertura sin pasar por el jugador



Para solucionar este problema se ha creado un sistema donde en ciertos lugares del mapa el NavMesh se actualiza cada cierto tiempo, además de que hay un NavMeshModifierVolume que sigue al jugador y hace que las zonas cercanas al jugador tengan un mayor peso para los enemigos a distancia.

CAMBIOS A LA API

Nodos y Clases nuevas:

Opposite Perception: Se ha creado un tipo de perception (que hereda de las percepciones compuestas) que niega la percepción “hija” que se le pase.

Dicho de otra forma, si la percepción hija devuelve *true*, la opposite perception devolverá *false* y viceversa.

Esto permite hacer comprobaciones inversas sin la necesidad de duplicar funciones. Aunque con la API sin modificaciones se puede usar el decorador Inverter para un funcionamiento similar, las OppositePerception se pueden usar para Máquinas de Estados o para casos donde se quiera hacer una comprobación negada en un condicional sin invertir el posible resultado del resto del árbol.

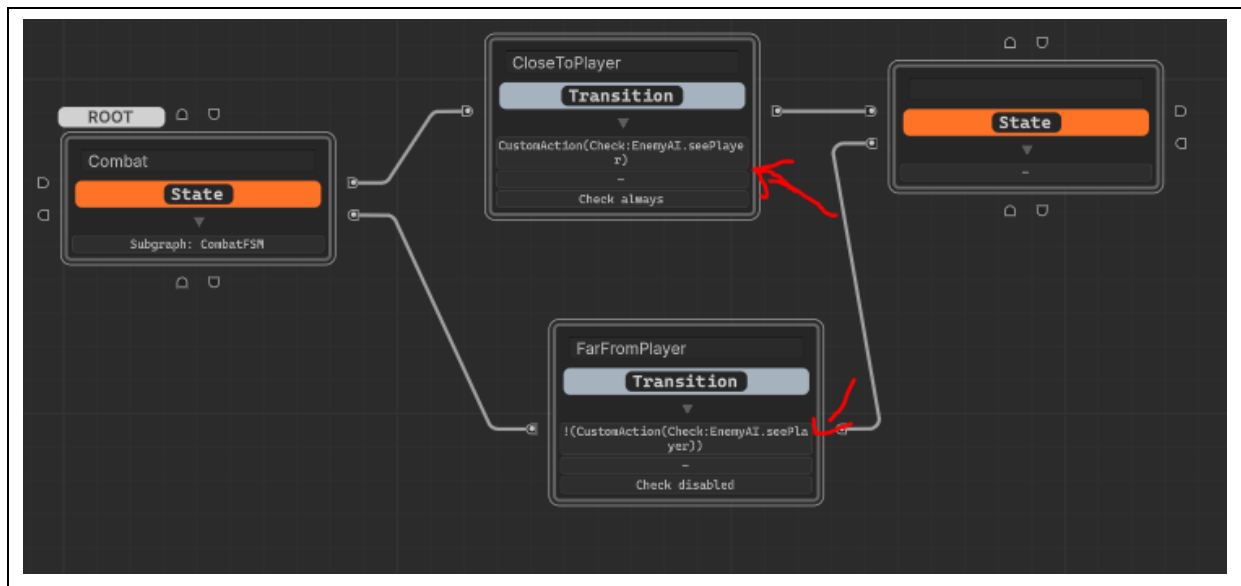
```
10 referencias
public override bool Check()
{
    if (Perceptions.Count == 0) return false;
    return !Perceptions[0].Check();
}
```

A pesar de que deriva de las percepciones compuestas, por lo que potencialmente podría tener varias percepciones, se ha añadido un bool *allowMultiple* a todos los nodos compuestos para que algunos de ellos solo muestren el botón de añadir si tienen 0 percepciones.

```
if (subPerceptionProperty == null || compoundPerception.compoundPerception.allowMultiple() || subPerceptionProperty.arraySize == 0)
{
    if (GUILayout.Button("Add element", EditorStyles.popup))
    {
        SearchWindow.Open(new SearchWindowContext(GUIUtility.GUIToScreenPoint(Event.current.mousePosition)),
            ElementCreatorWindowProvider.Create<PerceptionCreationWindow>((pType) => AddSubPerception(subPerceptionProperty, pType));
    }
}
```

Solo se muestre el botón add Element si se permiten múltiples percepciones o si no hay ninguna actualmente.

Ej: Para la máquina de estados superior (que comprueba si se está en modo patrulla o combate) no ha hecho falta crear una función *closeToPlayer* y *farFrom Player* para cada transición si no que podemos usar *seePlayer* y *!seePlayer*.



Reactive Branch Node: Clase abstracta que hereda del nodo Branch Node con la diferencia de que la condición de Branch se calcula cada Update, permitiendo cambiar de nodo sin que tenga que acabar la ejecución del árbol. De esta manera los Personajes pueden actuar de manera más reactiva con el entorno sin necesidad de sobre complicar los árboles.

Se le han añadido parámetros para que la comprobación no se haga cada ciclo de Update, y en ese caso la opción de elegir cada cuantos *frames* se va a hacer.

Además, se utiliza un numero aleatorio para que, en caso de que haya varios Personajes con el mismo árbol, sus comprobaciones no se hagan en el mismo *frame*.

Ej: Si hay 3 Shooters y la comprobación está configurada para hacerse cada 20 *frames*, uno lo hará en los *frames* 3,23,43..., otro en los *frames* 7,27,47...

Wait Time Frames	20
Sync Update	☐

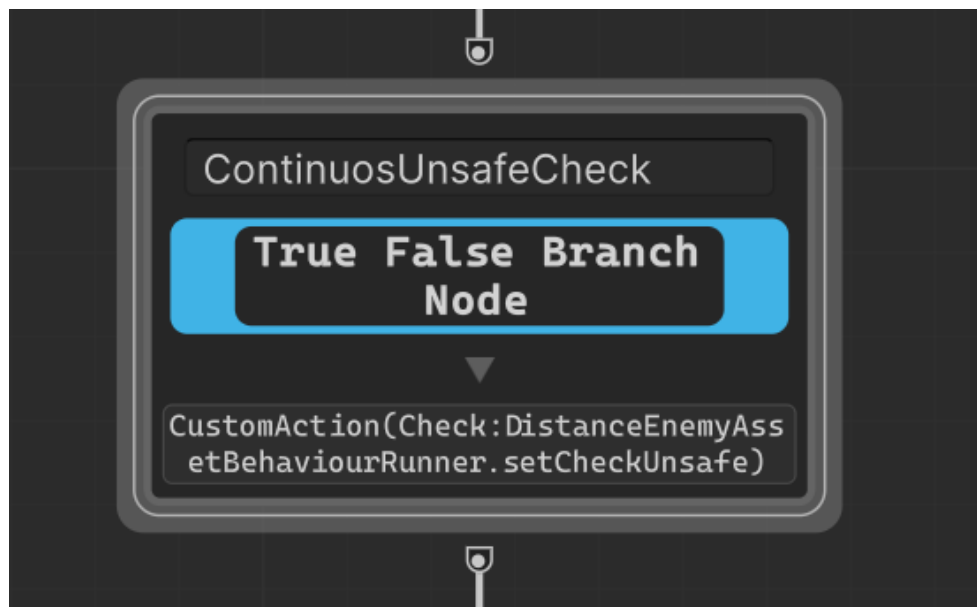
Wait Time Frames determina cada cuantos frames se hace. Sync Update da la opción de sincronizar o no todos los nodos.

```
wait = waitTimeFrames > 0;
randomDelay = syncUpdate ? 0 : Random.Range(0,10);
```

```
2 referencias
protected override Status UpdateStatus()
{
    if (!wait || (Time.frameCount + randomDelay) % waitTimeFrames == 0)
    {
        int branchIndex = SelectBranchIndex();
    }
}
```

True False Branch Node: Hereda de Reactive Branch Node y decide entre dos nodos hijo con una función que devuelve un booleano. Sirve para que un Personaje pueda alternar dinámicamente entre dos modos según una condición.

En el caso del juego se suele usar para que cuando una acción para de ser posible, el personaje se asegura de acabar todas sus tareas antes de cancelar la acción. Sin tener que añadir acciones que solo son necesarios en ciertos momentos en las transiciones de la máquina de estados.



```
3 referencias
protected override int SelectBranchIndex()
{
    bool index = Perception.Check();
    return index ? 1 : 0;
}
```

Ej 1: El enemigo shooter comprueba constantemente si la cobertura desde la que dispara es segura. En caso de que pare de serlo, no cambia al estado de correr directamente, sino que espera a haber disparado (por si ya estaba disparando), devuelve su mutex de ataque y ya después cambia al estado de huir.

Ej 2: Cuando el jugador sale del rango de cualquier enemigo melee (no explicados en esta.

Threshold Curve: Un tipo de curva para Utility Sytem que devuelve 0 si el input es menor a cierto valor y 1 si es mayor (o al revés si se marca como invertida).

Esto ya podía hacerse con curvas sigmoides (con un crecimiento muy alto) pero esta nueva clase es más fácil de entender y hace que los esquemas sean más legibles.

Modificaciones a clases existentes:

Se ha hecho una pequeña modificación al nodo de Branch Paralelo, específicamente a su opción de terminar cuando alguno de sus hijos termine en un estado específico (finish on success y finish on failure).

En el código original, para evitar comprobaciones de más, no se llamaba el update de hijos que ya habían terminado (ya que en circunstancias normales un nodo termina durante su update) pero esto hacía que si un nodo se completaba mediante una percepción PUSH nunca acabara el nodo paralelo.

```
while(currentChildId < m_children.Count && returnedStatus == Status.Running)
{
    var child = m_children[currentChildId];
    if(child.Status == Status.Running)
    {
        child.OnUpdated();
        currentChildStatus = child.Status;
        if (currentChildStatus == Status.Running) anyChildRunning |= true;

        if (finishOnSuccess && currentChildStatus == Status.Success || finishOnFailure && currentChildStatus == Status.Failure)
        {
            returnedStatus = currentChildStatus;
        }
    }
    currentChildId++;
}
```

Código original: se ignoran los nodos que ya han terminado.

Para solucionarlo, simplemente se forzó que las comprobaciones se hicieran, aunque ya hubiera terminado el nodo.

```
while(currentChildId < m_children.Count && returnedStatus == Status.Running)
{
    var child = m_children[currentChildId];
    UnityEngine.Debug.Log(child.Status);
    if (child.Status == Status.Running)
    {
        child.OnUpdated();
        UnityEngine.Debug.Log(child.Status);
        currentChildStatus = child.Status;
        if (currentChildStatus == Status.Running) anyChildRunning |= true;

        if (finishOnSuccess && currentChildStatus == Status.Success || finishOnFailure && currentChildStatus == Status.Failure)
        {
            returnedStatus = currentChildStatus;
        }
    }
    else
    {
        currentChildStatus = child.Status;
        if (finishOnSuccess && currentChildStatus == Status.Success || finishOnFailure && currentChildStatus == Status.Failure)
        {
            returnedStatus = currentChildStatus;
        }
    }
    currentChildId++;
}
```

Código actual: si el hijo ha acabado se comprueba si debe terminar la ejecución.

RESULTADOS



Los

pollos corren a la cobertura más cercana

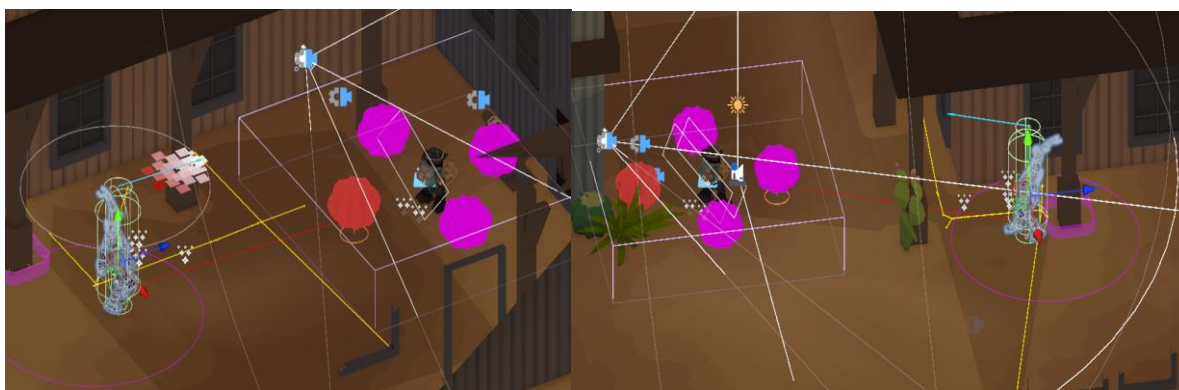


Mutex: Solo un enemigo (arriba a la izquierda) puede atacar a la vez, el resto esperan o se defienden.

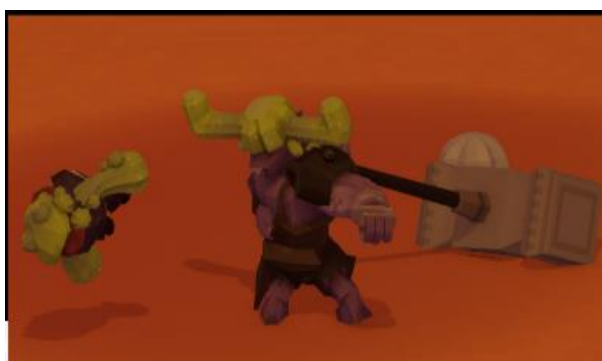


Los puntos del Kung-Fu Circle se mantienen siempre en la misma posición, sin importar a donde mire el jugador.

Los enemigos van cada uno a un punto, rodeando al jugador.



Si el jugador se mueve el objetivo del enemigo cambiará en tiempo real (indicado por la línea roja).



El enemigo bomba puede ser lanzado por el jugador y por el heavy

Esto solo ocurre en la zona circular del Nivel 1.

Una funcionalidad que se tenía pensado pero que no se ha podido llevar a cabo es hacer que el Pollo tuviera en cuenta si el camino hacia una cobertura pasa cerca del jugador antes de elegir a que cobertura ir. No como ahora que tiene en cuenta si la cobertura es segura, si desde la cobertura puede disparar al jugador y en ciertos lugares el camino una vez decidida la cobertura se hace rodeando al jugador.



El Boss hace ataques a distancia y a melee, dependiendo de su fase actual



El jefe hace el ataque Learn esquivable por ambas direcciones, pero guarda hacia donde ha esquivado el jugador.



El boss hace un ataque central seguido de una dirección dependiendo de donde ha esquivado más el jugador.

REPARTO DE TAREAS

Diego Pablo Fernández Manso: Diseño IA, Programación de la IA

Antonio Bernal de Celis: Diseño IA, Programación de Gameplay, Música y Sonido

Andrea Luengo Zazo: Programación de Gameplay

Candela Jiménez González: Diseño y modelado 3D, Animación 3D, Concept Art, Diseño de Ui

David del Castillo Enríquez: Game Design, guion y Level Design

Eduardo Almarza Blasco: Diseño y modelado 3D, Animación 3D, Arte técnico, Concept Art